# A6 Recursion & Iteration

**Modern Code**
**MC (MTD.ba VZ)**
**SS 2025**

Submission via Moodle — **24 Points**

This assignment focuses on recursion, iteration, and understanding when to use each approach. You will implement recursive solutions, compare recursion with iteration, and learn optimization techniques like memoization. Debugging skills will be naturally integrated as you trace recursive calls and understand call stacks.

**Submission Requirements:**

- Read the `Organizational Guidelines on Moodle`, and adhere to them!
- Submit a PDF document containing your **solution idea, code, and tests for each exercise**
- Include all Java source files (.java) in a ZIP archive
- As tests include copies or screenshots of the console output
- Use the naming convention: `MC_UE06_LASTNAME.pdf` and `MC_UE06_LASTNAME.zip`

**Important:** This assignment emphasizes understanding recursive thinking, base cases, and recursive cases. For each exercise, think about:

- What is the base case that stops the recursion?
- How does the recursive case make the problem smaller?
- When is recursion more natural than iteration?
- How can memoization optimize recursive functions?

**Hint:** Use the debugger to trace recursive calls and visualize the call stack. This will help you understand how recursion works and debug any issues.

**Exercise 1 (6 points) Fibonacci in Reverse: Iterative, Recursive, and Memoized**

Implement three different approaches to print the Fibonacci sequence in reverse order (from position n down to 0): iterative, recursive, and recursive with memoization.

1. Implement an iterative solution to calculate Fibonacci numbers
2. Implement a recursive solution to calculate Fibonacci numbers (without memoization)
3. Implement a recursive solution with memoization
4. For memoization, allow setting the memo space size. If a value exceeds the memo size, calculate it without memoization (graceful degradation)
5. Compare all three approaches in your solution idea
6. Test with different sequence positions (e.g., 0, 8, and a larger value like 15 to demonstrate graceful degradation)

**Hint:** For the recursive approaches, think about how you can achieve this in only ONE function with only ONE call from the main method (e.g. not just calling the recursive function within a for loop).

**Expected Output:**

```
=== Fibonacci: same for each: Iterative, Recursive, and Memoized ===
F(8) = 21
F(7) = 13
F(6) = 8
F(5) = 5
F(4) = 3
F(3) = 2
F(2) = 1
F(1) = 1
F(0) = 0
```

**Exercise 2 (5 points) Tower of Hanoi**

Implement the classic Tower of Hanoi problem using recursion. Make the number of discs configurable.

1. Implement a recursive solution for Tower of Hanoi
2. Make the number of discs a parameter (test with 3, 4, and 5 discs)
3. Display each move step-by-step (e.g., ”Move disc from A to C”)
4. Explain the recursive strategy in your solution idea
5. Use the debugger to trace the recursive calls and understand the three-peg logic

**Expected Output:**

```
=== Tower of Hanoi ===

========================================
Solving Tower of Hanoi with 3 disc(s)
========================================
Move disc from A to C
Move disc from A to B
Move disc from C to B
Move disc from A to C
Move disc from B to A
Move disc from B to C
Move disc from A to C
Total moves: 7
```

**Exercise 3 (5 points) Decimal to Binary Conversion**

Implement a recursive function to convert decimal numbers to binary representation.

1. Implement recursive decimal to binary conversion
2. Handle edge cases (0, 1)
3. Test with various decimal numbers
4. Explain the recursive strategy in your solution idea

**Expected Output:**

```
=== Decimal to Binary Conversion ===

Decimal: 0    Binary: 0
Decimal: 1    Binary: 1
Decimal: 5    Binary: 101
Decimal: 10    Binary: 1010
Decimal: 15    Binary: 1111
Decimal: 42    Binary: 101010
Decimal: 255    Binary: 11111111
```

**Exercise 4 (5 points) Number to Words**

Implement a recursive function to convert numbers to their word representation (e.g., 1234 ”One Thousand Two Hundred Thirty Four”).

1. Implement recursive number to words conversion
2. Handle numbers up to thousands (0-9999)
3. Handle edge cases (0, single digits, teens, tens, hundreds, thousands)
4. Test with various numbers
5. Explain the recursive strategy in your solution idea

**Expected Output:**

```
=== Number to Words Conversion ===

0     Zero
5     Five
19     Nineteen
23     Twenty Three
42     Forty Two
100     One Hundred
123     One Hundred Twenty Three
456     Four Hundred Fifty Six
1000     One Thousand
1234     One Thousand Two Hundred Thirty Four
5678     Five Thousand Six Hundred Seventy Eight
9999     Nine Thousand Nine Hundred Ninety Nine
```

**Exercise 5 (3 points) Tribonacci**

Implement the Tribonacci sequence recursively. Tribonacci is similar to Fibonacci but uses three base cases: $T(0) = 0$, $T(1) = 0$, $T(2) = 1$, and $T(n) = T(n-1) + T(n-2) + T(n-3)$ for $n > 2$.

1. Implement recursive Tribonacci sequence
2. Handle the three base cases: T(0) = 0, T(1) = 0, T(2) = 1
3. Test with various sequence positions
4. Explain the recursive strategy in your solution idea

**Expected Output:**

```
=== Tribonacci Sequence ===
T(0) = 0
T(1) = 0
T(2) = 1
T(3) = 1
T(4) = 2
T(5) = 4
T(6) = 7
T(7) = 13
T(8) = 24
T(9) = 44
T(10) = 81
T(11) = 149
T(12) = 274
T(13) = 504
T(14) = 927
```

---